idea-harness

Product Requirements Document

Version: 1.1
Status: Product Baseline
Supersedes: PRD v1.0 and prior addenda

---

1. Product Definition

idea-harness is a local-first AI product-development control plane that helps a user take an idea from ambiguity to verified software using a coordinated team of Staff-level AI experts.

The product begins with:

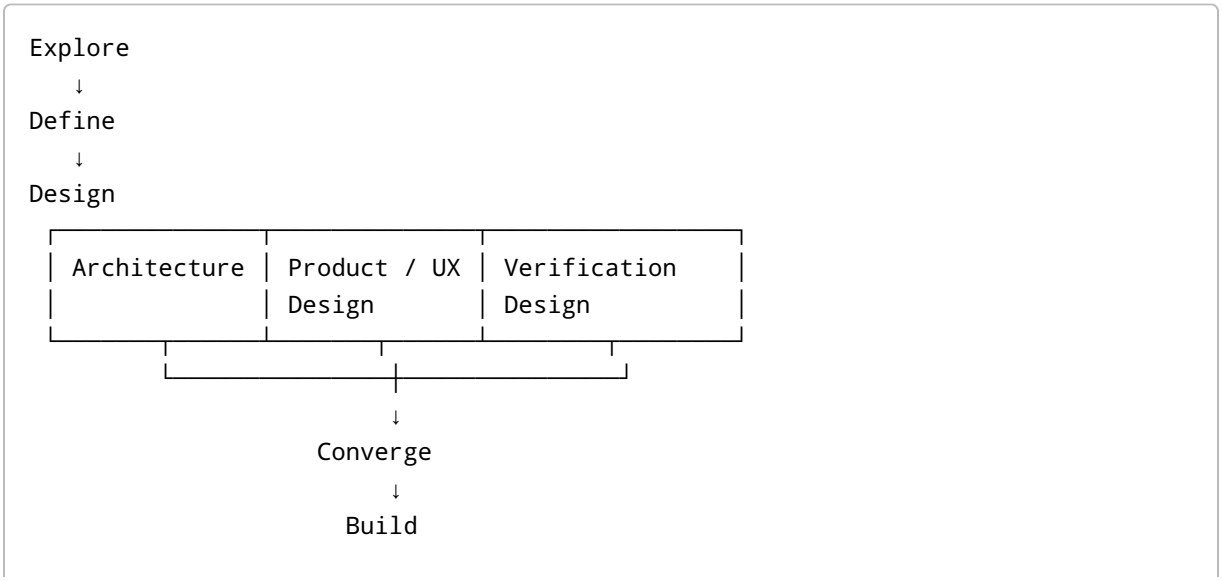
> I have an idea.

not:

> I have a Git repository.

It coordinates research, product thinking, design, architecture, quality, security, planning, implementation, review, and verification while preserving enough evidence and provenance to explain why important decisions and implementation choices exist.

The default lifecycle is:



```
                    ↓
                 Verify
                    ↓
                  Done
```

High-risk initiatives dynamically introduce specialists such as Security, Privacy, Database, SRE, Developer Experience, Payments, or AI Security.

The user should experience this as:

```
I provide direction.
The system explores.
The experts challenge each other.
The system synthesizes.
I make the important calls.
The system plans.
Agents execute.
Reviewers challenge the work.
Evidence proves the result.
```

The product must not feel like managing a Kanban board full of AI agents.

---

2. Product Thesis

idea-harness is not primarily:

- a coding agent
- an AI chat interface
- a Kanban application
- a project-management tool
- a Git wrapper
- an agent framework

It is a **product-development operating system and control plane**.

Its durable value is the Product Graph connecting:

```
Evidence
   ↓
Assertions
   ↓
Requirements
   ↓
```

```
Decisions
   ↓
Plans
   ↓
Tasks
   ↓
ChangeSets
   ↓
Tests
   ↓
Verification
```

The board, documents, reviews, and agent runtime are interfaces onto this underlying graph.

---

3. North-Star Experience

The initial experience should be:

```
mkdir something-new
cd something-new

idea init
idea dev

idea new "Build an API that ..."
```

The user should not need:

- Git
- a remote repository
- a project skeleton
- prior product documentation
- agent configuration expertise
- workflow-engine knowledge

idea-harness should establish the initial understanding of the idea and propose an appropriate development path.

The user sees:

```
CURRENT UNDERSTANDING

We're building ...
```

```
Primary user
...

Core value
...

Known constraints
...

Still unresolved
• ...
• ...


RECOMMENDED NEXT MOVE

Resolve X before implementation.

Why
Architecture and UX both depend on it.

[Review decision]
```

The system complexity should remain available one layer deeper.

---

4. Product Principles

4.1 Calm on the surface, rigorous underneath

Routine usage should expose:

- the current idea
- what has been learned
- what is unresolved
- what needs human judgment
- what changed
- what happens next

The machinery behind these outcomes should be progressively disclosed.

4.2 Start before Git

Supported starting conditions:

- empty folder
- folder containing notes
- existing codebase without Git
- local Git repository
- remote-connected repository

Git is optional infrastructure.

4.3 Decisions over agent conversations

The system should coordinate experts for the user.

When experts disagree, idea-harness should synthesize the conflict into a human decision surface.

Bad:

```
Chat with PM
Chat with Architect
Chat with Designer
```

Good:

```
DECISION NEEDED

Should sharing links expire?

Product recommends:
Optional expiration

Security recommends:
Expiration by default

UX recommends:
No expiration unless selected

Orchestrator recommendation:
30 days by default, with "Never" available

[Accept]
```

```
[Choose alternative]
[Discuss]
```

4.4 Staff-level by contract

Role quality must come from:

- explicit competencies
- required behaviors
- artifact requirements
- review rubrics
- regression evals

not merely prompts saying "act like a Staff PM."

4.5 Adversarial by default

The development loop is:

```
propose
   ↓
challenge
   ↓
revise
   ↓
approve
   ↓
execute
   ↓
challenge again
```

Every material execution plan is independently reviewed.

Every source ChangeSet is independently reviewed.

No agent may approve its own work.

Review depth may vary by risk.

4.6 Risk determines ceremony

A documentation edit and an authentication migration must not receive identical process depth.

Risk controls:

- review breadth
- specialist involvement
- verification breadth
- model quality
- human approvals
- context breadth
- rollback requirements

Review remains mandatory.

4.7 Deterministic evidence beats probabilistic judgment

If something can be conclusively checked by:

- compiler
- test
- type checker
- linter
- schema validator
- security scanner
- deterministic command

that evidence should be preferred over LLM judgment.

4.8 Humans make consequential judgments

The system should optimize human attention for:

- ambiguous product tradeoffs
- accepted risk
- irreversible changes
- high-impact decisions
- unresolved expert disagreement

It should not create approval fatigue.

4.9 Everything meaningful should be explainable

Users should be able to ask:

```
What is this?
Why does this exist?
Why did this happen?
What depends on this?
Where did this come from?
What changed?
```

Product concepts:

```
Explain
Why
Impact
Trace
What Changed
```

---

5. Primary User

Initial target:

- technical founder
- founder who codes
- staff engineer
- technical product leader
- independent developer
- very small AI-native product team

The first product must be exceptional for one sophisticated user working locally.

Multi-user collaboration is not required for v0.1.

---

6. Primary Jobs to Be Done

JTBD 1

When I have an idea, help me turn it into a coherent product definition without requiring me to manually orchestrate AI agents.

JTBD 2

When there are unresolved assumptions or tradeoffs, surface the decisions that genuinely require my judgment.

JTBD 3

When agents design or implement something, ensure they challenge weak assumptions and follow my standards.

JTBD 4

When implementation changes source code, isolate and independently review that change before accepting it.

JTBD 5

When requirements change, tell me which decisions, plans, tests, and source changes may now be invalid.

JTBD 6

When I return after hours or days, tell me what changed and what the current understanding is.

JTBD 7

When something fails, explain the cause, impact, and available recovery path.

---

7. User-Facing Information Model

Users should primarily interact with:

```
Idea
Current Understanding
Open Questions
Decisions
Requirements
Artifacts
Tasks
Changes
Reviews
Verification
```

Lower-level implementation concepts remain available for debugging:

```
StageRun
Run
ContextBundle
RunManifest
Operation
GraphEdge
Executor
Blob digest
```

---

8. Current Understanding

Every Initiative maintains a synthesized projection called **Current Understanding**.

It represents the most current approved view of:

- what is being built
- target user
- problem
- desired outcome
- major constraints
- important decisions
- unresolved assumptions
- open questions
- major risks

It is not an independent authoritative artifact.

It is derived from approved Product Graph state.

Example:

```
CURRENT UNDERSTANDING

We're building a local-first tool that extracts product
decisions from meeting transcripts and preserves their
source provenance.

Primary user
Product teams running frequent cross-functional meetings.

Core value
Prevent decisions from disappearing into meeting history.
```

```
Key decisions
• Extracted decisions remain linked to transcripts.
• Human corrections override AI extraction.
• Slack ingestion is deferred.

Still unresolved
• Conflict resolution semantics
• Data-retention policy
```

Acceptance Criteria

- Current Understanding updates after approved material changes.
- Draft or rejected agent work cannot silently modify it.
- Each statement can be traced to its authoritative source.
- Users can inspect why a statement appears.

---

9. Recommended Next Move

The orchestrator should continuously maintain a suggested highest-value next action.

Example:

```
RECOMMENDED NEXT MOVE

Resolve decision conflict semantics before implementation.

Why
Architecture, storage, and UX all depend on this behavior.

Affected
Architecture
Product Design
Verification

[Review decision]
```

The recommendation must be derived from the workflow and Product Graph, not invented as generic advice.

---

10. What Changed

The system must provide semantic change summaries.

Example:

```
SINCE YOU LAST LOOKED

PRODUCT
• REQ-08 changed from SHOULD to MUST.
• Mobile support became an explicit non-goal.

DESIGN
• Passkeys replaced session-based authentication.

BUILD
• Three ChangeSets were accepted.
• One ChangeSet was rejected after authorization review.

QUALITY
• Integration coverage was added for REQ-08.
```

This should be available via UI and CLI.

```
idea changed
```

---

11. Development Phases

The human-facing workflow is:

```
Explore
Define
Design
Build
Verify
```

These are macro-phases, not raw workflow nodes.

Explore

Contains:

- idea framing
- research
- evidence gathering
- assumption identification
- open-question discovery

Define

Contains:

- product requirements
- goals
- non-goals
- metrics
- acceptance criteria
- product decisions

Design

May run in parallel:

- architecture
- product design
- UX
- verification design
- threat modeling
- domain specialist review

Build

Contains:

- execution planning
- task decomposition
- implementation
- deterministic checks
- ChangeSet review

Verify

Contains:

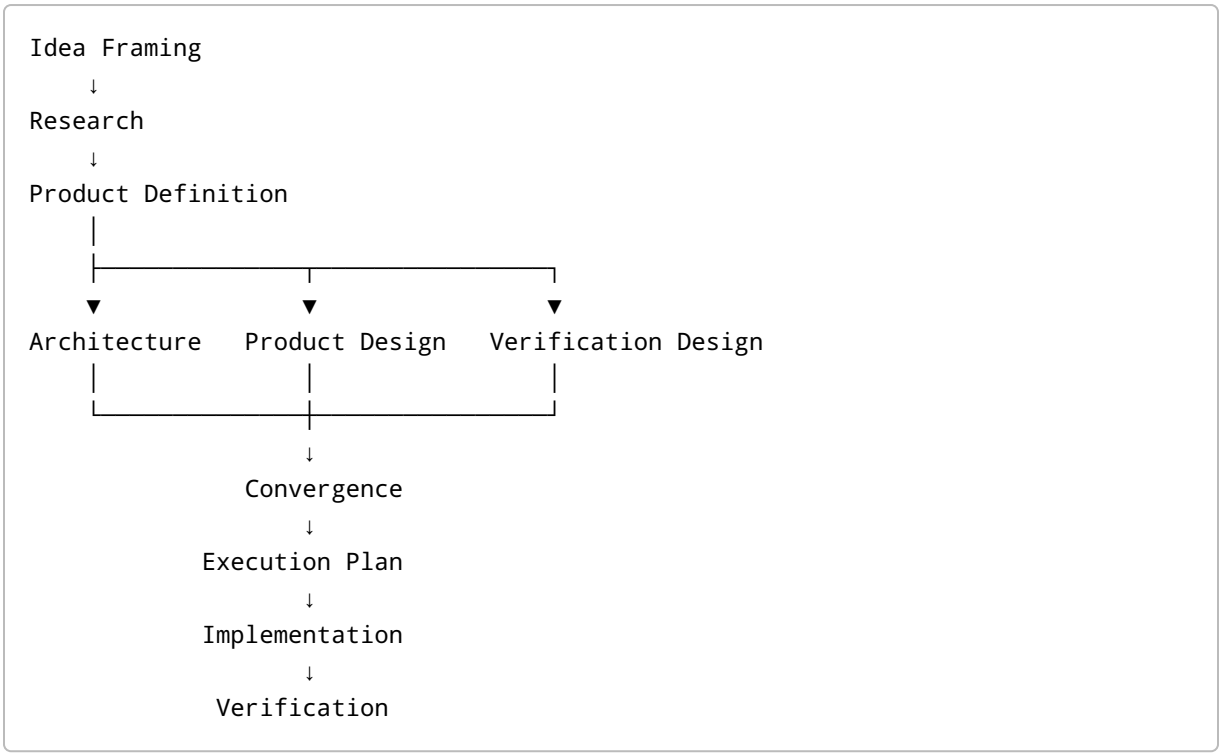
- acceptance verification
- security verification
- UX verification
- operational readiness where relevant
- final requirement coverage

---

12. Workflow Model

The underlying engine is a DAG.

Example:



An Initiative may have multiple active StageRuns simultaneously.

---

13. Assertions

Facts, assumptions, and hypotheses are first-class structured assertions.

Kinds:

```
Fact
Assumption
Hypothesis
```

Statuses:

```
Candidate
Accepted
Disputed
Retracted
Superseded
```

Assertions support:

- confidence
- evidence
- human correction
- downstream impact analysis

Model memory is not evidence.

---

14. Research and Evidence

Research claims must trace to sources.

Sources should record:

- source type
- URL/path
- publisher
- retrieval time
- captured content where appropriate
- content digest
- trust level

The system must surface conflicting evidence.

External content is untrusted input.

---

15. Requirements

Requirements are structured objects rendered into readable PRDs.

Each Requirement contains:

- stable ID
- version
- statement
- priority
- status
- acceptance criteria
- evidence/decision relationships

Requirement versions preserve logical identity when wording evolves without changing intent.

A material intent change creates a new Requirement.

---

16. Acceptance Criteria

Acceptance criteria explicitly specify how they can be verified whenever possible.

Verification types:

```
Automated test
Command
Schema
Security review
UX review
Agent review
Human
```

Example:

```
AC-17-1

Revoked API keys return HTTP 401.

Verification:
Automated integration test
```

---

17. Decisions

Decisions are first-class, immutable after approval.

Kinds may include:

- Product
- Architecture
- Design
- Security
- Operations

Product Decision Records and Architecture Decision Records are UI projections of the same Decision model.

Changing an approved decision creates a new Decision that supersedes the prior one.

---

18. Product Graph

The system maintains typed relationships between:

```
Source
Assertion
Requirement
Acceptance Criterion
Decision
Artifact
Task
ChangeSet
Test
Review
Verification
```

The graph enables:

```
idea trace REQ-17
idea impact REQ-17
idea why TASK-42
```

The graph is architecturally central but visually secondary.

Users should consume its value primarily through:

- Why
- Impact
- Trace
- stale indicators
- semantic diffs
- requirement coverage

A raw graph view remains available for experts.

---

19. Artifacts

Built-in artifacts include:

```
IdeaBrief
ResearchBrief
PRD
TechnicalDesign
DesignSpec
VerificationPlan
ThreatModel
ExecutionPlan
OperationalReadiness
VerificationReport
```

Documents must feel readable while exposing their structured components.

Artifacts have immutable versions and explicit lineage.

---

20. Semantic Diff

The system must compare artifact revisions semantically.

Example:

```
REQUIREMENT ADDED

REQ-14
Sharing links must be revocable.
```

```
REQUIREMENT CHANGED

REQ-08

BEFORE
Links SHOULD expire.

AFTER
Links MUST support configurable expiration.


DECISION SUPERSEDED

ADR-4 → ADR-9

Redis
→
PostgreSQL advisory locks
```

Raw source diff remains available.

---

21. Staleness

When upstream information changes:

```
Current
Potentially Stale
Stale
```

are separate from approval/status.

The system:

1. traverses affected dependencies
2. marks candidates potentially stale
3. performs impact analysis
4. classifies each as:
5. unaffected
6. revalidate
7. revise

8. obsolete

The system should avoid unnecessary regeneration.

---

22. Core Expert Team

Built-in roles:

Research Analyst

Evidence gathering, claim quality, competitive/context research.

Staff Product Manager

Problem framing, scope, requirements, metrics, tradeoffs.

Staff Product Designer

Interaction design, information architecture, states, accessibility.

Staff UX Researcher capability

Mental models, usability assumptions, research methodology.

Staff Software Architect

Architecture, system boundaries, APIs, failure modes.

Staff Engineering Lead

Implementation strategy and task decomposition.

Staff Quality Engineer

Verification strategy before implementation.

Staff Security Engineer

Threat modeling and application/agent security.

Adversarial Reviewer

Independent challenge.

Product Orchestrator

Coordination, synthesis, impact analysis, routing.

---

23. Dynamic Specialists

Deterministic triggers may invoke specialists.

Examples:

Signal	Specialist
PII	Privacy
Authentication	Security
Schema migration	Database
Distributed system	Distributed Systems
User-facing UI	Accessibility
Performance-sensitive	Performance
Deployable service	SRE
Public API/SDK	DevEx
Payments	Payments
AI/agent feature	AI/AI Security

Agents may recommend additional expertise.

The orchestrator must authorize the consultation.

---

24. Role Contracts

Roles define:

- competencies
- required behaviors
- artifact requirements
- required checks
- review rubrics
- eval suite

The system must ship regression evals for built-in Staff-level roles.

---

25. Quality Before Implementation

Quality is involved during Design.

The Quality Engineer creates a Verification Plan before the execution plan is finalized.

It should challenge vague acceptance criteria such as:

```
fast
easy
secure
scalable
intuitive
```

and make them more measurable where possible.

---

26. Security Before Convergence

High-risk product definitions may trigger threat modeling before architecture is approved.

Security may create additional requirements that enter the normal Product Graph.

Security is not merely an implementation review.

---

27. Risk

v0.1 uses explicit tiers:

```
Low
Medium
High
Critical
```

Avoid fake numerical precision.

Signals include:

- authentication
- authorization
- payments
- PII
- migrations
- destructive operations
- public APIs
- cryptography
- production infrastructure
- dependency/supply-chain changes
- irreversibility

An expert may raise risk.

Automatic semantic reasoning should not silently lower deterministic risk.

---

28. Adversarial Review

Every material execution plan and every ChangeSet receives independent review.

Review findings contain:

- severity
- confidence
- category
- finding
- evidence
- impact
- recommendation

Severity:

```
Critical
High
Medium
Low
Observation
```

Confidence:

```
High
Medium
Low
```

Review depth scales with risk.

---

29. Review Convergence

Default maximum review cycles:

```
3
```

After that:

```
review_non_convergence
```

requires human judgment.

Findings may be:

```
Fixed
Accepted Risk
Not Applicable
Duplicate
Deferred
Rejected
```

Critical accepted risk requires human approval by default.

---

30. Review Economics

Review must not make the system unusably slow.

Low-risk review should use the smallest relevant context.

High-risk review may include additional specialists.

The system should track:

- review latency
- review cost
- findings raised
- false positives
- defect escapes

---

31. ChangeSet

A ChangeSet is the fundamental source-change primitive.

It exists independently of Git.

Flow:

```
Isolated worker workspace
        ↓
Source changes
        ↓
ChangeSet
        ↓
Deterministic checks
        ↓
Adversarial review
        ↓
Approved
        ↓
Canonical workspace
        ↓
Optional Git commit
```

Workers never directly mutate the canonical workspace.

---

32. Git

Git states:

```
None
Local
Remote
```

Git may appear at any point.

```
git init
```

must be detected automatically.

idea-harness must never automatically publish local source code.

---

33. Change Envelope

Execution plans identify expected scope.

Example:

```
Expected:
src/billing/**
tests/billing/**

Excluded:
src/auth/**
```

Unexpected source changes require explanation and may trigger re-review.

This prevents accidental scope creep.

---

34. Verification

Initiatives cannot complete while mandatory acceptance criteria remain unverified.

Verification evidence should be traceable to:

- requirement
- acceptance criterion
- test/check/review
- ChangeSet where relevant

The product should explicitly surface uncovered requirements.

---

35. Skills, Tools, Executors, Policies, Workflows

Separate extension categories:

Skill

Non-executable expertise, templates, rubrics, schemas.

Tool

Executable external capability.

Executor

Agent/coding runtime.

Policy Pack

Declarative standards and rules.

Workflow Pack

Declarative methodology.

Skills may come from local or pinned Git sources in v0.1.

Skills cannot execute arbitrary code.

---

36. Security Requirements

Hard product invariants:

- default-deny capabilities
- security enforcement outside models
- no self-review
- no silent permission escalation
- no silent model fallback
- external content untrusted
- unreviewed agent output lower trust

- secrets excluded/redacted from persistent output
- cross-project isolation
- sandboxed execution by default
- source changes reversible before acceptance
- serialized canonical workspace mutation

---

37. Autonomy

Presets:

```
Observe
Supervised
Standard
High
```

They modify human-approval policy, not security invariants.

The exact policy implications must be visible.

---

38. Primary UX

Global navigation:

```
Ideas
Decisions
Activity

Runtime
Settings
```

Ideas

Portfolio board and Initiative Home.

Decisions

Human judgment and blocking decisions.

Activity

What changed and chronological history.

Runtime

Expert/operator view of agents and execution.

Settings

Models, roles, workflows, skills, policies, security.

---

39. Initiative Home

The default screen for developing one idea.

Primary hierarchy:

1. Current Understanding
2. Recommended Next Move
3. Decisions / unresolved questions
4. What Changed
5. Current Work
6. Artifacts/Requirements/Changes one level deeper

The user should not land on raw workflow metadata.

---

40. Decisions UX

A Decision surface should synthesize expert disagreement.

Users should usually make one product decision rather than coordinate multiple AI conversations.

Decisions should include:

- question
- alternatives
- expert perspectives
- recommendation
- rationale
- consequences
- affected areas

41. Board

The board is primarily for multiple initiatives.

Macro columns:

```
Explore
Define
Design
Build
Verify
Done
```

Cards show detailed sub-state.

Avoid percentage progress.

Prefer:

```
Architecture      ✓
Product Design    •
Verification      ✓
Security          !
```

42. Runtime

Runtime is an operator interface.

Show:

```
role
model
executor
current action
phase
elapsed time
cost
```

```
task
recent activity
```

Do not stream internal chain-of-thought.

Show operational state and tool activity.

---

43. Why / Explain / Impact / Trace

These are first-class interactions throughout the UI.

Examples:

```
Why is Security involved?
Why is this blocked?
Why is this artifact stale?
Why was this model chosen?
What depends on REQ-17?
Where did this requirement come from?
```

---

44. Accessibility

UI initiatives must always consider:

- keyboard navigation
- focus
- semantics
- screen readers
- contrast
- target size
- error identification
- responsive behavior
- loading states
- empty states
- permission states

Accessibility cannot disappear because a project forgot to request it.

---

45. Empty State

The initial UI should ask:

> What are you thinking about building?

Single input:

```
Describe the idea, feature, problem, or change...
```

Possible starting contexts:

```
Start from an idea
Start from an existing folder
Start from a requirement
Start from a bug
```

---

46. Existing Project Onboarding

Example:

```
I found an existing TypeScript application.

React
Fastify
PostgreSQL
Vitest
Git repository

✓ Architecture analyzed
✓ Test commands detected
! No engineering standards found

[Use defaults]
[Review setup]
```

Greenfield should clearly acknowledge that no code or Git exists yet.

---

47. Performance and Speed

The product must optimize perceived and actual speed.

Use:

```
deterministic checks
→ cheap semantic evaluation
→ deeper model review
→ specialist review
→ human judgment
```

UI should show useful progress state rather than endless spinners.

---

48. Debugging and Reliability

The system must support:

```
idea explain
idea why
idea impact
idea trace
idea changed
idea inspect run
idea inspect context
idea doctor
```

No completed workflow transition may disappear after a restart.

---

49. Product Metrics

Track:

```
idea-to-reviewed-PRD
idea-to-first-code
decision latency
ChangeSet review latency
requirements coverage
verification coverage
```

```
review false-positive rate
defect escape rate
human intervention rate
human override rate
cost per initiative
context-token efficiency
agent/provider failure rate
```

More agents are not automatically better.

These metrics should determine whether sophistication improves outcomes.

---

50. v0.1 Delivery

Milestone 1

Idea → reviewed PRD

Milestone 2

PRD → reviewed execution plan

Milestone 3

Plan → safe reviewed ChangeSet

Milestone 4

Requirement → verified product

Build vertical slices rather than the entire platform horizontally.

---

51. v0.1 Non-Goals

Do not initially build:

- hosted SaaS
- multi-user collaboration
- enterprise RBAC
- remote worker fleet
- skill marketplace

- extension registry
- mobile app
- deployment platform
- CI/CD replacement
- full semantic code index
- organization-wide memory
- complex numeric risk engine
- large executor ecosystem

---

52. Final Product Definition

idea-harness is a local-first product-development control plane that takes an idea, builds and continuously updates a shared understanding of the problem, assembles the appropriate Staff-level experts, exposes the consequential decisions that require human judgment, creates traceable product and technical plans, safely delegates execution, independently reviews every proposed source change, verifies the implementation against its requirements, and can explain why every important decision and change exists.

The defining user experience is:

```
Show me the idea.

Show me what we've learned.

Show me what changed.

Show me what still isn't known.

Show me what needs my judgment.

Show me what happens next.

Hide the machinery until I ask for it.
```